INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA, E TECNOLOGIA DE SÃO PAULO

Alice Peters Gonçalves do Prado Levy
Beatriz Sati Minoura
Clarice Cortez Generoso
Heitor Borges Pinto

Projeto Integrador: Sabor & Clic

Sistema de Gestão para Dark Kitchens e Espaços Gastronômicos

São Paulo
2026

Alice Peters Gonçalves do Prado Levy
Beatriz Sati Minoura
Clarice Cortez Generoso
Heitor Borges Pinto

Projeto Integrador: Sabor & Clic

Sistema de Gestão para Dark Kitchens e Espaços Gastronômicos

Disciplinas de Artes Visuais Digitais, Banco de Dados,
Desenvolvimento Web, Laboratório de Programação e
Segurança e Desenvolvimento de Software, 2° ano 231
Instituto Federal, Campus São Paulo

Professores Celso Gonsalez Saes,
Claudete de Oliveira Alves, Daniella Zanellato,
Fabio Vieira do Amaral e Igor de Moraes Sampaio,

São Paulo
2026

Sumário

Sumário 3
1. INTRODUÇÃO 4
2. OBJETIVO 4
3. USUÁRIOS DO SISTEMA 4
3.1 Administrador 5
3.2 Cliente 5
3.3 Cozinheiro 5
4. TECNOLOGIAS UTILIZADAS 6
5. ARQUITETURA DO SISTEMA 6
6. DESENVOLVIMENTO DA INTERFACE 7
6.1 Tela inicial 8
6.2 Pedidos 8
6.3 Cardápio 9
6.4 Cozinha 9
6.5 Reservas 10
7. DESENVOLVIMENTO EM JAVASCRIPT 11
7.1 Carrinho de compras 11
8. DESENVOLVIMENTO EM PYTHON E FLASK 12
8.1 Model 12
9. CONTROLLERS E ROTAS 13
10. REGRAS DE NEGÓCIO 14
11. COMUNICAÇÃO ENTRE JAVASCRIPT E FLASK 15
12. ENDPOINTS 16
13. SEGURANÇA 16
14. PRÓXIMAS ETAPAS 16
15. CONCLUSÃO 17

1. INTRODUÇÃO

O projeto Sabor & Clic consiste no desenvolvimento de um sistema web destinado ao gerenciamento de uma dark kitchen ou espaço gastronômico.

O sistema tem como objetivo reunir, em uma única aplicação, funcionalidades relacionadas ao gerenciamento do cardápio, pedidos, cozinha, reservas e estoque, proporcionando uma organização mais eficiente das atividades do estabelecimento.

Ele é dividido em diferentes níveis de acesso, considerando as necessidades do administrador, cliente e cozinheiro.

Nesta primeira etapa do projeto, o desenvolvimento está concentrado na construção da interface, na implementação das funcionalidades utilizando JavaScript e na criação da estrutura inicial do backend utilizando Python e Flask. A implementação do banco de dados e a parte de proteção do sistema serão realizadas posteriormente.

---

2. OBJETIVO

O objetivo do projeto é desenvolver um sistema web para auxiliar no gerenciamento de estabelecimentos gastronômicos, permitindo organizar informações relacionadas aos produtos, pedidos, reservas, cozinha e estoque.

Nesta primeira etapa, busca-se desenvolver uma aplicação funcional utilizando HTML, CSS, JavaScript, JSON, Fetch API, Python e Flask, estabelecendo a comunicação entre o front-end e o back-end.

---

3. USUÁRIOS DO SISTEMA

O sistema possui três principais tipos de usuários:

3.1 Administrador

O administrador é responsável pelo gerenciamento do estabelecimento. Entre suas funções estão:

- Gerenciar o cardápio;
- Cadastrar e editar pratos;
- Alterar preços e descrições;
- Ativar ou desativar produtos;
- Acompanhar pedidos;
- Gerenciar reservas;
- Acompanhar o estoque;
- Visualizar informações e relatórios.

3.2 Cliente

O cliente utiliza o sistema para:

- Visualizar o cardápio;
- Consultar informações dos pratos;
- Adicionar produtos ao carrinho;
- Realizar pedidos;
- Acompanhar o status do pedido.

3.3 Cozinheiro

O cozinheiro utiliza o sistema para:

- Visualizar os pedidos recebidos;
- Acompanhar a fila de produção;
- Consultar os pedidos em preparo;
- Atualizar o status dos pedidos.

---

4. TECNOLOGIAS UTILIZADAS

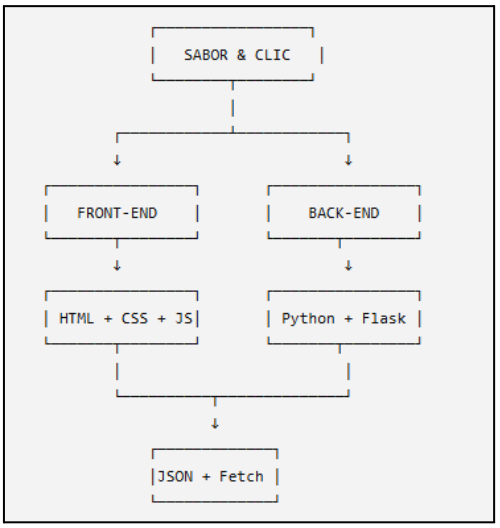
Para o desenvolvimento da primeira etapa foram utilizadas as seguintes tecnologias:

Tecnologia	Utilização
HTML5	Estrutura da interface
CSS3	Estilização e organização visual
JavaScript	Interatividade e manipulação da interface
Fetch API	Comunicação entre front-end e back-end
JSON	Formato de troca de informações
Python	Desenvolvimento do back-end
Flask	Framework utilizado no desenvolvimento do servidor

---

5. ARQUITETURA DO SISTEMA

A aplicação foi organizada de maneira a separar as responsabilidades entre o front-end e o back-end.



Legenda: Arquitetura de comunicação do Sabor & Clic.

- ➔ O front-end é responsável pela apresentação das informações e pela interação com o usuário, proporcionando uma interface visual organizada e facilitando o acesso às funcionalidades do sistema.
- ➔ O JavaScript realiza a manipulação da interface, controla as interações do usuário e é responsável pela comunicação com o servidor por meio de requisições.
- ➔ O Python/Flask é responsável pelo processamento das requisições recebidas, pela execução das regras de negócio e pelo tratamento dos dados antes de enviá-los de volta ao front-end.

Nesta etapa, os dados ainda são mantidos temporariamente em memória. O banco de dados será implementado em uma etapa posterior.

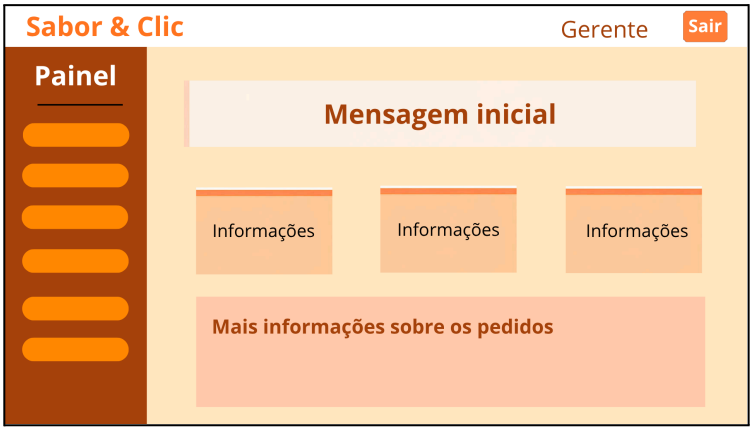
---

6. DESENVOLVIMENTO DA INTERFACE

A interface do Sabor & Clic foi desenvolvida buscando organizar as principais funcionalidades do sistema de maneira simples e acessível. Inicialmente, ela começou a ser desenvolvida no Figma, mas, devido à pouca familiaridade com a plataforma, optamos por utilizar o Canva para fazer a interface base do site. As imagens das interfaces abaixo foram desenvolvidas posteriormente no próprio HTML, CSS e JavaScript.

Entre as telas desenvolvidas estão:

- Tela inicial;
- Cardápio;
- Pedidos;
- Cozinha;
- Reservas;
- Estoque;
- Relatórios;
- Configurações.



Interface base desenvolvida no Canva

6.1 Tela inicial

A tela inicial apresenta informações resumidas sobre o funcionamento do estabelecimento, permitindo visualizar dados relacionados aos pedidos, reservas e demais informações importantes.



Figura 1 — Tela inicial do sistema Sabor & Clic.

6.2 Pedidos

A tela de pedidos permite acompanhar os pedidos realizados e seus respectivos status.

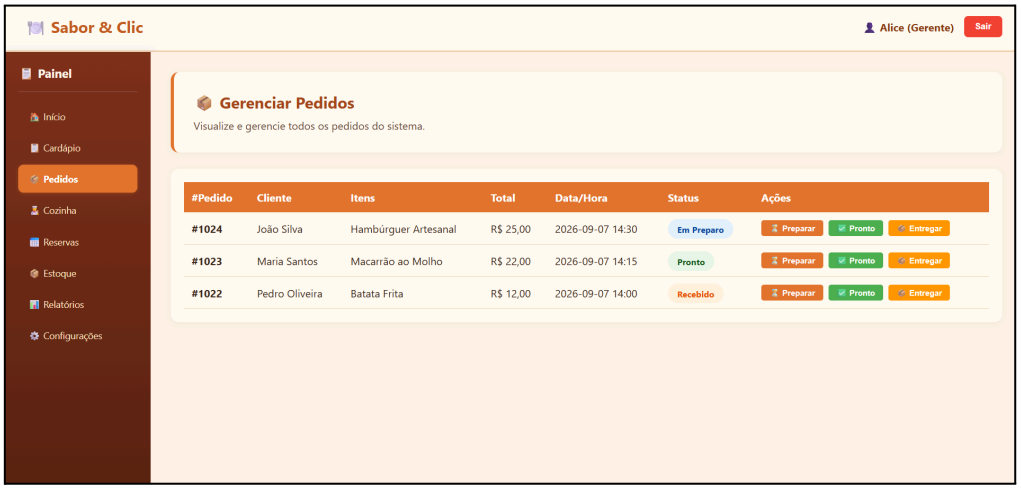


Figura 2 — Tela de gerenciamento de pedidos.

6.3 Cardápio

A tela de cardápio apresenta os produtos disponíveis para o cliente, contendo nome, descrição, categoria e preço. Também é possível adicionar os produtos ao carrinho para posteriormente realizar um pedido.



Figura 3 — Tela de cardápio.

6.4 Cozinha

A tela da cozinha apresenta os pedidos que precisam ser preparados, funcionando como um painel de acompanhamento da produção.

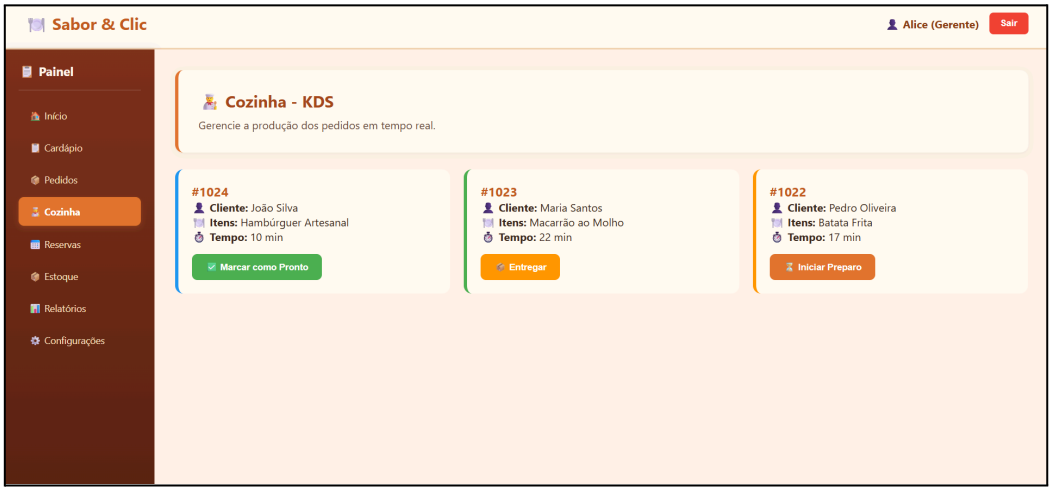


Figura 4 — Painel da cozinha.

6.5 Reservas

A tela de reservas permite visualizar e organizar as reservas realizadas no estabelecimento.

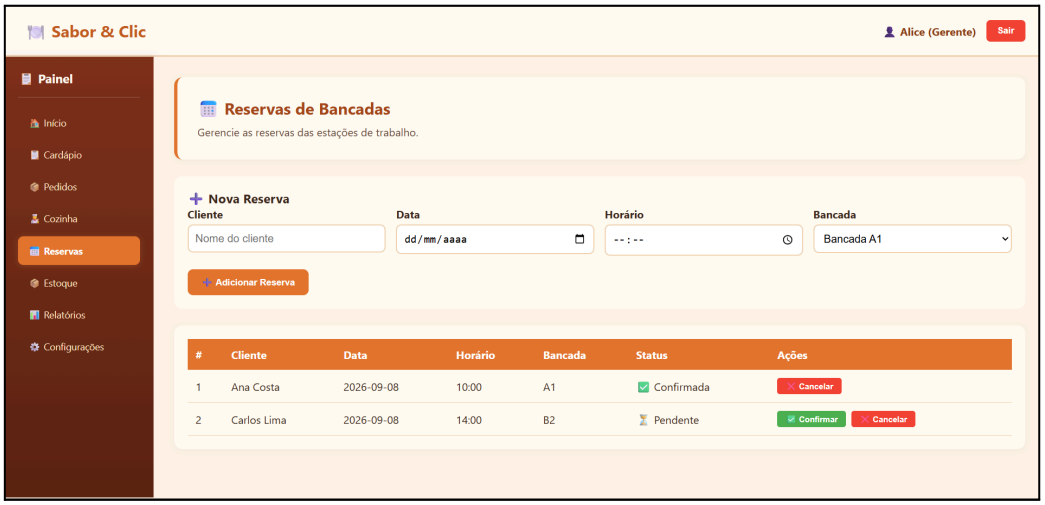


Figura 5 — Tela de reservas.

6.6 Paleta de cores

Para a cor principal do site foi definida a cor laranja, pois essa cor estimula o apetite, traz energia e dinamismo, mas sem a agressividade ou urgência excessiva do vermelho. Geralmente, costuma ser uma das principais cores para sites de restaurantes, bares e deliveries tradicionais.

Já para o fundo, o branco e o off-White foram escolhidos. Essas cores mantêm a interface limpa e moderna.



Legenda: Paleta de cores feita no Canva usada no site.

7. DESENVOLVIMENTO EM JAVASCRIPT

O JavaScript é utilizado principalmente para controlar a interação do usuário com a interface e realizar atualizações dinâmicas da página.

Foram utilizados recursos como:

- Arrays;
- Objetos;
- Funções;
- Manipulação do DOM;
- Template literals;
- Eventos.

E alguns comandos como:

- forEach();
- find();
- filter();
- map();
- reduce();
- push();
- unshift();

Os dados utilizados inicialmente no sistema incluem pratos, pedidos, reservas e informações de estoque.

7.1 Carrinho de compras

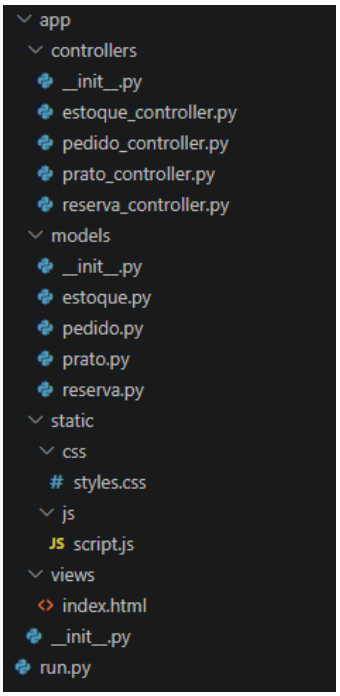
Foi implementado um sistema de carrinho que permite adicionar produtos, alterar suas quantidades e remover itens.

O carrinho é controlado pelo JavaScript enquanto o usuário interage com a interface.

Porém, o cálculo definitivo do pedido será realizado pelo backend, evitando que regras importantes do sistema dependam exclusivamente do código executado no navegador.

---

8. DESENVOLVIMENTO EM PYTHON E FLASK



O backend do sistema está sendo desenvolvido utilizando Python e Flask. A estrutura foi organizada seguindo o padrão MVC, separando os modelos e os controladores, como mostra a figura ao lado:

8.1 Model

Os models representam os dados utilizados pelo sistema.

Como exemplo, foi criado o modelo *Prato*:

```
class Prato:

    def __init__(self, id, nome, preco, descricao, categoria):
        self.id = id
        self.nome = nome
        self.preco = preco
        self.descricao = descricao
        self.categoria = categoria

    def to_dict(self):
```

```
        return {
            "id": self.id,
            "nome": self.nome,
            "preco": self.preco,
            "descricao": self.descricao,
            "categoria": self.categoria
        }
```

O método *to_dict()* permite transformar o objeto em um formato que pode ser convertido para JSON.

---

9. CONTROLLERS E ROTAS

Os Controllers são responsáveis por receber as requisições e executar as ações correspondentes.

Por exemplo, para consultar os pratos cadastrados foi criada a rota:

```
@prato_controller.route("/pratos", methods=["GET"])
def listar_pratos():

    return jsonify([
        prato.to_dict()
        for prato in pratos
    ])
```

Essa rota pode ser acessada por:

http://localhost:5000/pratos

O servidor retorna os pratos no formato JSON.

---

10. REGRAS DE NEGÓCIO

As regras de negócio definem como o sistema deve processar e validar as informações relacionadas às operações realizadas pelos usuários. No projeto, essas regras são concentradas principalmente no backend, desenvolvido em Python com Flask, garantindo que informações importantes não dependam apenas da interface do usuário.

O JavaScript é responsável pela interação com a interface e pela comunicação com a API, enquanto o Flask recebe as solicitações, valida os dados e realiza os processamentos necessários antes de retornar uma resposta.

Por exemplo, no processo de realização de um pedido, o fluxo ocorre da seguinte maneira:

[Cliente] → [JavaScript] → [Fetch — POST /pedidos] → [Flask] → [Validação] → [Cálculo do total] → [Criação do pedido] → [JSON] → [JavaScript] → [Atualização da interface]

Dessa forma, o navegador não é responsável por decidir sozinho informações importantes do pedido. O backend realiza as validações e os cálculos necessários, contribuindo para que as regras do sistema sejam aplicadas de maneira consistente, independentemente da interface utilizada pelo usuário.

Entre as principais regras de negócio previstas para o sistema estão:

- Os dados enviados para a realização de um pedido devem ser validados pelo backend.
- O valor total do pedido deve ser calculado pelo servidor com base nos produtos e respectivas quantidades.
- Um pedido somente deve ser criado após a validação dos dados necessários.
- As informações processadas pelo backend devem ser retornadas em formato JSON para que o JavaScript possa atualizar a interface.
- O JavaScript não deve ser a única camada responsável por validar informações importantes para o funcionamento do sistema.

---

11. COMUNICAÇÃO ENTRE JAVASCRIPT E FLASK

A comunicação entre o front-end e o back-end é realizada utilizando a Fetch API e o formato JSON.

Por exemplo, o JavaScript pode solicitar os pratos ao Flask:

```
async function carregarPratos() {

    const resposta = await fetch("/pratos");

    const pratos = await resposta.json();

    console.log(pratos);
}
```

O Flask recebe a solicitação e retorna os dados:

```
@prato_controller.route("/pratos", methods=["GET"])
def listar_pratos():

    return jsonify([
        prato.to_dict()
        for prato in pratos
    ])
```

Assim, o JavaScript não precisa manter todos os dados diretamente no código. Ele pode solicitar as informações ao backend sempre que necessário.

---

12. ENDPOINTS

Nesta primeira etapa, os principais endpoints planejados/desenvolvidos são:

Método	Endpoint	Função
GET	/	Verificar o funcionamento do sistema
GET	/pratos	Listar os pratos
GET	/pedidos	Listar pedidos
POST	/pedidos	Criar um novo pedido

Novas rotas poderão ser adicionadas conforme o desenvolvimento das funcionalidades de reservas, estoque e gerenciamento do sistema.

---

13. SEGURANÇA

A segurança será implementada progressivamente durante o desenvolvimento do projeto.

Nesta etapa, busca-se manter a separação entre o código executado no navegador e as regras processadas pelo servidor.

Nas próximas etapas serão implementados recursos relacionados à autenticação, controle de acesso, validação e sanitização dos dados.

---

14. PRÓXIMAS ETAPAS

Após a conclusão desta primeira etapa, serão desenvolvidas novas funcionalidades para ampliar o sistema.

Entre elas estão:

- Integração com banco de dados;
- Persistência dos dados;
- Operações completas de CRUD;

- Controle de estoque integrado aos pedidos;
- Autenticação dos usuários;
- Controle de acesso por função;
- Validação e sanitização;
- Melhorias na comunicação entre front-end e back-end.

---

15. CONCLUSÃO

O desenvolvimento inicial do Sabor & Clic permitiu estabelecer a estrutura do sistema e desenvolver suas principais interfaces.

A utilização de JavaScript possibilitou criar uma interface dinâmica e interativa, enquanto o Python com Flask estabeleceu a estrutura inicial do backend.

A separação entre front-end e back-end também permite que as regras de negócio sejam concentradas no servidor, facilitando a futura integração com o banco de dados.

Na segunda etapa do projeto, serão implementados recursos adicionais no back-end, incluindo middleware, gerenciamento de sessões e cookies, autenticação e controle de acesso, além da integração com o banco de dados.

Dessa forma, a primeira etapa estabelece uma base para a evolução do sistema nas próximas fases do projeto.

Para ter acesso aos códigos que fizemos até agora, basta acessar o seguinte link do GitHub: https://github.com/Alicelevy/Projeto-Sabor-Click